

ADAPTIVE THRESHOLD CALIBRATION IN INDUSTRIAL QUALITY ASSURANCE USING CLASSICAL MACHINE LEARNING

Refia Nur Şenyiğit - 231805065

1. Introduction

In modern manufacturing industries, automated quality control systems are very important for checking product dimensions and reducing errors. Computer vision systems with cameras are widely used on conveyor belts to inspect parts [1]. However, these systems need stable lighting to work correctly. Changes in factory lighting change the pixel brightness in images. This makes standard, fixed thresholding methods fail because object boundaries blend into the background. Global methods like Otsu's thresholding also fail when the lighting is not uniform across the image.

To solve this problem, this project creates a lightweight, real-time machine learning system to predict the best threshold value (0-255) using simple image statistics. Instead of using heavy Convolutional Neural Networks (CNNs) that require expensive hardware and create delays (latency), we use feature engineering and classical regression models [2].

Our dataset is completely unique and new. It contains grayscale images taken directly from a real production line using an *Intel RealSense D455 Depth* [3] industrial camera. Because the camera and background never change, the only variance comes from lighting shifts. This makes classical machine learning the best and fastest solution for this real-time industrial task.

2. Literature Review

Image segmentation and threshold selection are well-studied topics in computer vision. Otsu's method is a popular standard because it selects a threshold by separating pixel groups. However, academic studies show that global thresholding methods do not work well when the background lighting has reflections or shadows.

With Industry 4.0, using machine learning for pixel-level tasks has become common practice. For small industrial computers (edge devices), extracting statistical data from histograms—like mean, variance, skewness, and kurtosis—is a fast and effective alternative to heavy deep learning models. Many researchers use Support Vector Regression (SVR) and Tree-based models (like Random Forests) to map these histogram features directly to operational values. Following the **CRISP-DM (Cross-Industry Standard Process for Data Mining)** framework, this project connects simple image processing with adaptive machine learning to build a low-latency system for automated inspection.

3. Methodology

This project follows the **CRISP-DM** process model to ensure a structured and clear workflow.

3.1. Data Understanding

3.2. Data Preparation and Feature Engineering

To make the models run fast in production, we do not use raw images. Instead, we extract a 14-dimensional feature vector from each image histogram:

- **Basic Statistics:** mean, median, and standard deviation (std).
- **Shape Features:** skewness (asymmetry of the histogram) and kurtosis (how sharp the histogram peaks are).
- **Percentiles:** p10, p25, p75, p90, p95, and p99 to understand extreme light changes.
- **Domain Features:** dominant_bin (the most frequent pixel value group), standard otsu value, and brightness ratios (bright_ratio_100, bright_ratio_150, bright_ratio_200) which show the percentage of bright pixels.

3.3. Modeling Strategy

We selected **five different machine learning algorithms** from the scikit-learn library to compare their performance:

1. **Linear Regression:** Our baseline model.
2. **Decision Tree Regressor:** A model that splits data based on feature rules.
3. **Random Forest Regressor:** An ensemble model that combines multiple decision trees to prevent overfitting.
4. **K-Nearest Neighbors (KNN):** A model that finds the closest historical data points.
5. **Support Vector Regression (SVR):** A model that maps features into a higher dimension to find a non-linear decision boundary.

3.4. Performance Evaluation Methods

To evaluate the models correctly and prevent overfitting, we used three methods:

- **5-Fold Cross-Validation:** To check model stability across different data splits.
- **Hold-out Split:** A standard train-test split to measure Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R² Score.
- **Three-Way Split:** Splitting the data into Training, Validation, and Test sets to simulate a real factory deployment.

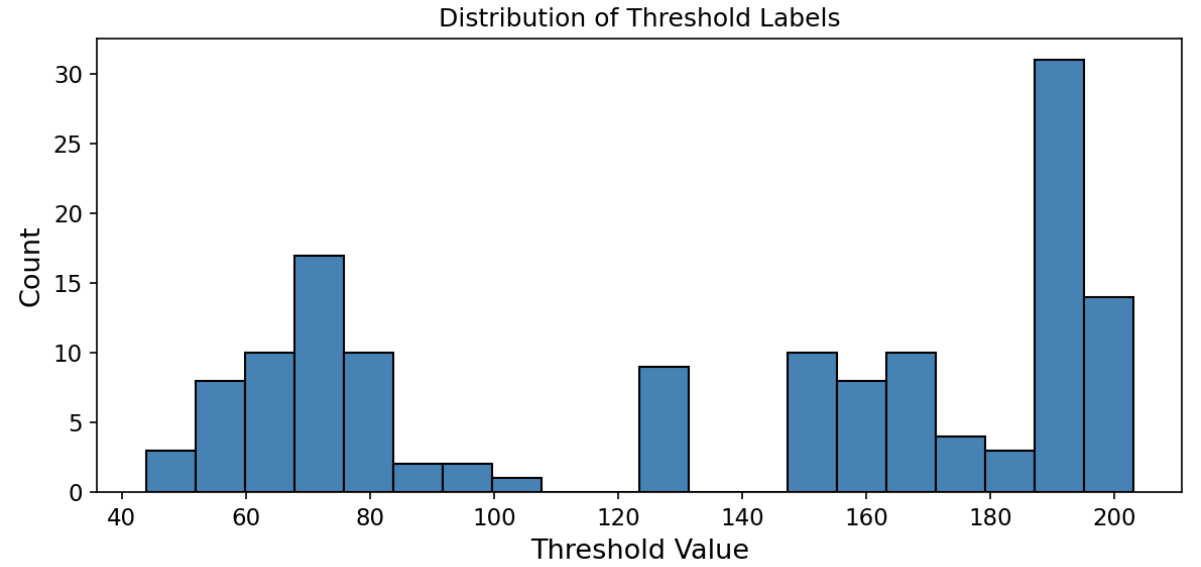
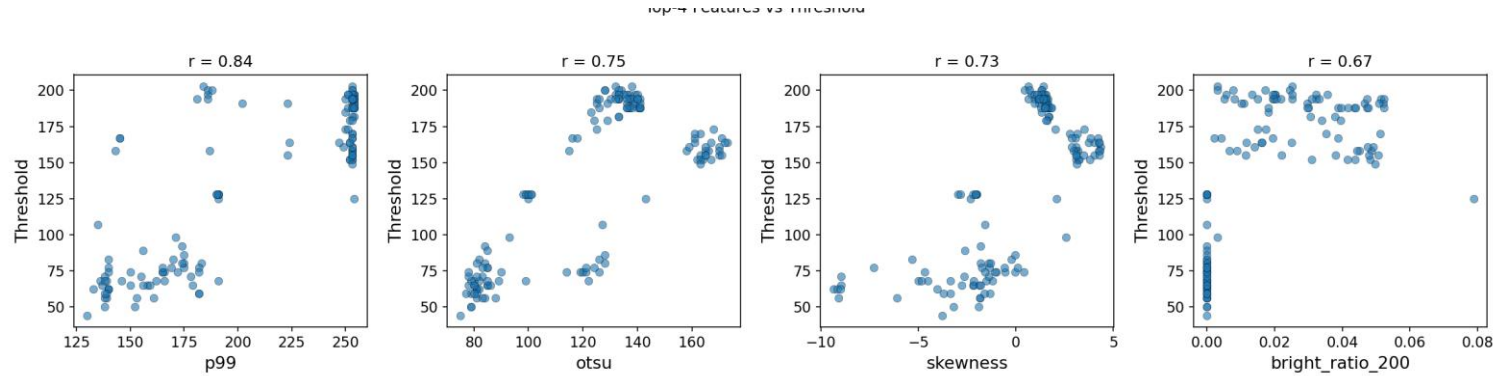
4. Findings & Results

4.1. Statistical Analysis and Correlation

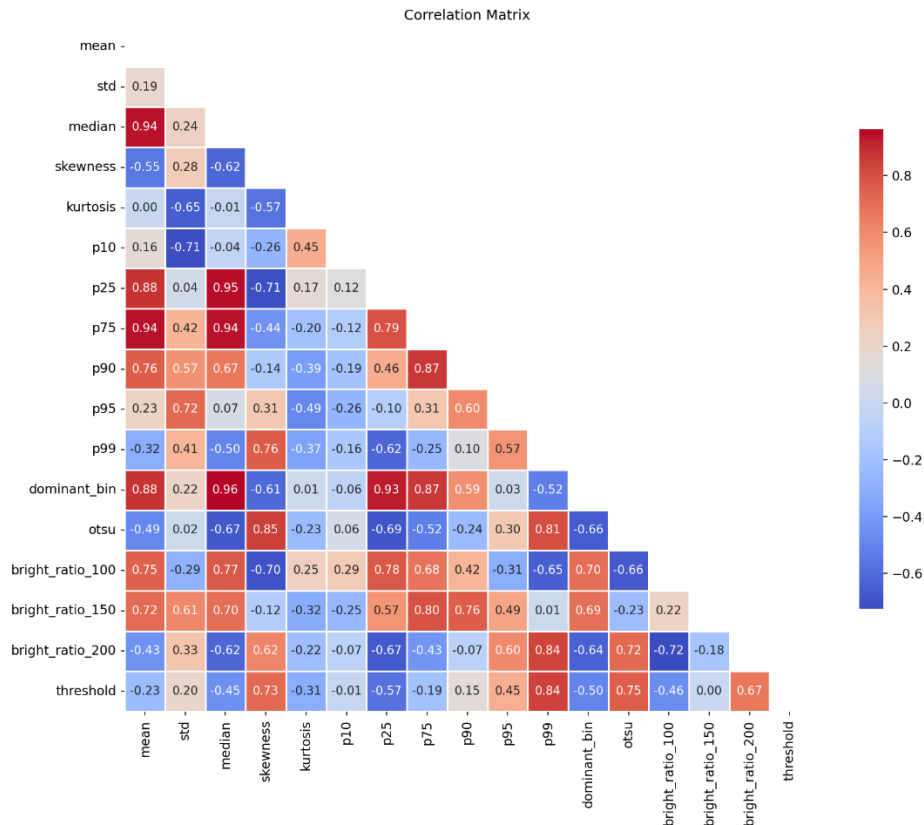
The engineered features show a strong relationship with the target threshold. Pearson correlation analysis shows that high-intensity features are the most important.

Feature	Pearson Correlation (r) with Threshold
p99	0.8390
otsu	0.7525
skewness	0.7252
bright_ratio_200	0.6709
p25	-0.5748
dominant_bin	-0.5040

The high positive correlation of p99 (0.839) shows that the brightest pixels in the image heavily determine the correct threshold.



Standard otsu only has a 0.752 correlation with the ideal threshold, which explains mathematically why standard Otsu thresholding failing in factory setup.

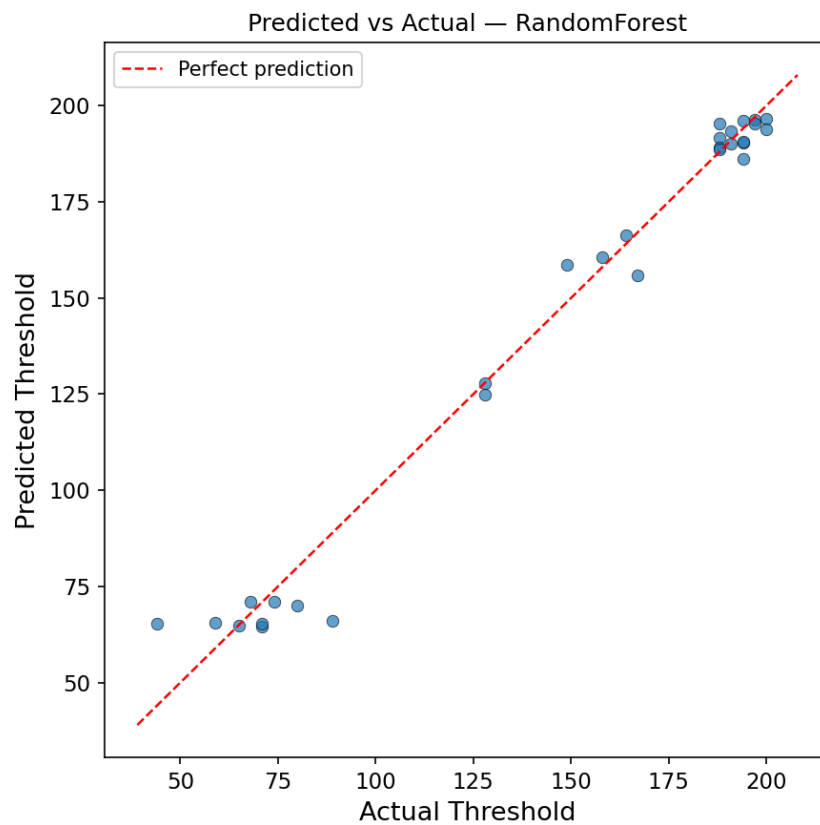
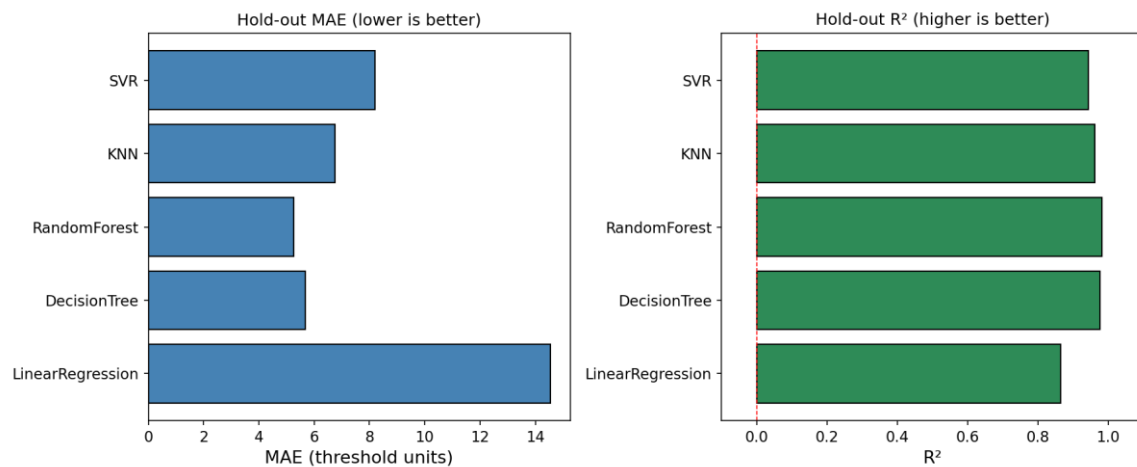


4.2. Comparative Model Evaluation

The results show clear differences between the five models across the evaluation methods.

Full Performance Table

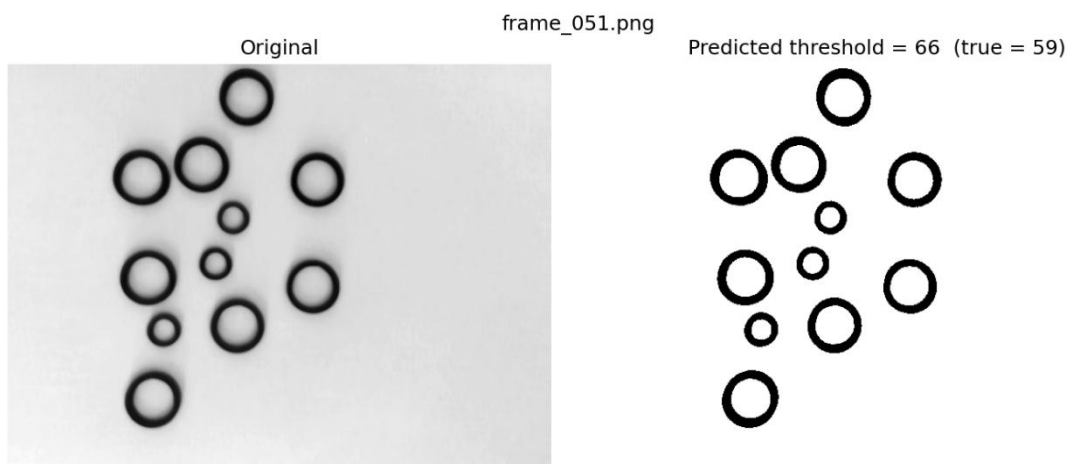
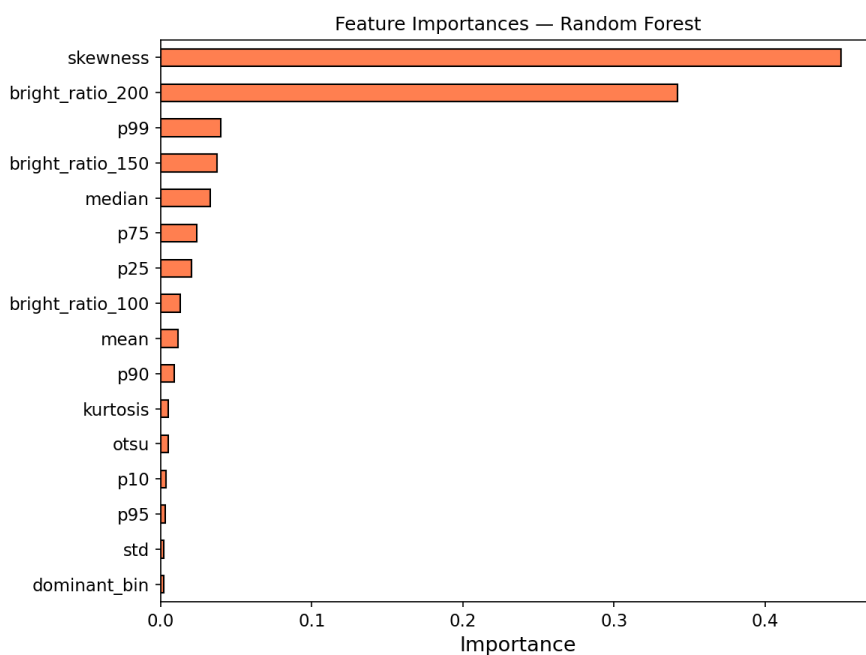
Algorithm	Hold-out MAE	Hold-out RMSE	Hold-out R2	5-Fold CV MAE	CV MAE Std (σ)	Test MAE (3-Way)	Test R2 (3-Way)
Linear Regression	14.535	20.281	0.864	18.846	4.620	12.459	0.886
Decision Tree	5.690	8.393	0.977	7.512	1.950	11.276	0.757
Random Forest	5.255	7.599	0.981	7.560	2.101	11.894	0.752
KNN	6.745	10.652	0.962	8.615	1.127	6.683	0.961
SVR	8.203	13.191	0.942	9.462	1.424	9.132	0.889



4.3. Discussion of Results

- **Linear Regression:** This model had the worst performance (CV MAE = 18.84). This proves that the relationship between histogram statistics and the optimal threshold is non-linear.

- **Tree Models in Hold-out: Random Forest** achieved the lowest error in the standard hold-out test ($MAE = 5.25$, $R^2 = 0.981$). It learns percentile features very well.
- **Overfitting in Tree Models:** When we look at the **Three-Way Split**, Random Forest and Decision Tree errors increase on the completely unseen Test set (Test $MAE \sim 11.2 - 11.8$, and R^2 drops to ~ 0.75). This indicates a slight overfitting problem.
- **The Best Model (KNN): K-Nearest Neighbors (KNN)** showed the best ability to generalize. In the Three-Way Split, it achieved an excellent **Test MAE of 6.683 and a Test R^2 of 0.961**. It also had the lowest variance during cross-validation ($\sigma = 1.127$). Because our camera and background are fixed, lighting changes form clear clusters in the feature space, which allows KNN to make very accurate and stable predictions.



5. Discussion and Conclusion

5.1. Interpretation

The results clearly show that classical machine learning using simple histogram features can replace complex deep learning models for specific industrial tasks. By extracting key features like p99, otsu, and skewness, the KNN model can predict the correct threshold with a very small error (~6 grey levels). This meets the real-time requirements of the production line perfectly.

5.2. Limitations

The primary limitation of this study is the small dataset size (142 images). While it is enough for classical regression models, collecting more images with extreme lighting changes (like high reflections or deep shadows) would make the system more robust.

5.3. Future Work

For future improvements, we recommend:

- **Testing More Models:** Using lightweight gradient boosting algorithms like LightGBM [7] or XGBoost.[8]
- **Continuous Learning:** Creating a loop that automatically collects and adds new lighting images to the KNN database over time.
- **Hardware Deployment:** Exporting the KNN model into C++ binaries using ONNX Runtime to run directly on the camera hardware with microsecond latency.

6. Acknowledgments

I want to thank the course instructor. And mostly to the CEO of Elmosis Automation Company – Serkan ANLAK for his supports.

7. References

1. Liu, B., Zhang, Q., Xue, L., Song, P. X.-K., & Kang, J. (2024). Robust High-Dimensional Regression with Coefficient Thresholding and Its Application to Imaging Data Analysis. *Journal of the American Statistical Association*, 119(545), 715-729. <https://doi.org/10.1080/01621459.2022.2142590>
2. Shi, X., Misch, D., & Vranjes-Wessely, S. (2023). A comprehensive assessment of image processing variability in pore structural investigations: Conventional thresholding vs. machine learning approaches. *Gas Science and Engineering*, 115, 205022. <https://doi.org/10.1016/j.jgsce.2023.205022>
3. <https://www.realsenseai.com/products/real-sense-depth-camera-d455f/>
4. *Documentation scikit-learn: Machine learning in Python—Scikit-learn 0.21.3 documentation*. (n.d.). Retrieved June 10, 2026, from <https://scikit-learn.org/0.21/documentation.html>
5. *OpenCV: OpenCV modules*. (n.d.). Retrieved June 10, 2026, from <https://docs.opencv.org/4.13.0/>
6. *pandas documentation—Pandas 3.0.3 documentation*. (n.d.). Retrieved June 10, 2026, from <https://pandas.pydata.org/docs/>

7. Zhou, Y., Wang, W., Wang, K., & Song, J. (2022). Application of LightGBM Algorithm in the Initial Design of a Library in the Cold Area of China Based on Comprehensive Performance. *Buildings*, 12(9), 1309. <https://doi.org/10.3390/buildings12091309>
8. Ren, X., Guo, H., Li, S., Wang, S., & Li, J. (2017). A Novel Image Classification Method with CNN-XGBoost Model. In C. Kraetzer, Y.-Q. Shi, J. Dittmann, & H. J. Kim (Eds.), *Digital Forensics and Watermarking* (pp. 378–390). Springer International Publishing. https://doi.org/10.1007/978-3-319-64185-0_28
9. Bharati, M. H. (2002). *Multivariate Image Analysis and Regression for Industrial Process Monitoring and Product Quality Control*. <http://hdl.handle.net/11375/6173>
10. *Evaluation of image processing technique in identifying rice blast disease in field conditions based on KNN algorithm improvement by K-means—Larijani—2019—Food Science & Nutrition—Wiley Online Library*. (n.d.). Retrieved June 10, 2026, from <https://onlinelibrary.wiley.com/doi/full/10.1002/fsn3.1251>
11. Patel, A. K., Chatterjee, S., & Gorai, A. K. (2019). Development of a machine vision system using the support vector machine regression (SVR) algorithm for the online prediction of iron ore grades. *Earth Science Informatics*, 12(2), 197–210. <https://doi.org/10.1007/s12145-018-0370-6>
12. Agarwal, C., & Sharma, A. (2011). Image understanding using decision tree based machine learning. *ICIMU 2011 : Proceedings of the 5th International Conference on Information Technology & Multimedia*, 1–8. <https://doi.org/10.1109/ICIMU.2011.6122757>
13. Dong, X., Taylor, C. J., & Cootes, T. F. (2021). A Random Forest-Based Automatic Inspection System for Aerospace Welds in X-Ray Images. *IEEE Transactions on Automation Science and Engineering*, 18(4), 2128–2141. <https://doi.org/10.1109/TASE.2020.3039115>
14. Yang, J., & Lee, C.-H. (2025). Real-Time Data-Driven Method for Bolt Defect Detection and Size Measurement in Industrial Production. *Actuators*, 14(4), 185. <https://doi.org/10.3390/act14040185>
15. Liu, X., Zhao, D., Xiong, R., Ma, S., Gao, W., & Sun, H. (2011). Image Interpolation Via Regularized Local Linear Regression. *IEEE Transactions on Image Processing*, 20(12), 3455–3469. <https://doi.org/10.1109/TIP.2011.2150234>
16. Lee, J. (1978). *Digital Image Processing by Use of Local Statistics*. (No. NRL8192). Article NRL8192. <https://apps.dtic.mil/sti/html/tr/ADA053843/>
17. Kumar, G., & Bhatia, P. K. (2014). A Detailed Review of Feature Extraction in Image Processing Systems. *2014 Fourth International Conference on Advanced Computing & Communication Technologies*, 5–12. <https://doi.org/10.1109/ACCT.2014.74>
18. Humeau-Heurtier, A. (2019). Texture Feature Extraction Methods: A Survey. *IEEE Access*, 7, 8975–9000. <https://doi.org/10.1109/ACCESS.2018.2890743>
19. Wang, A., Wang, Y., & Chen, Y. (2019). Hyperspectral image classification based on convolutional neural network and random forest. *Remote Sensing Letters*, 10(11), 1086–1094. <https://doi.org/10.1080/2150704X.2019.1649736>
20. Huang, T. S., Schreiber, W. F., & Tretiak, O. J. (1971). Image processing. *Proceedings of the IEEE*, 59(11), 1586–1609. <https://doi.org/10.1109/PROC.1971.8491>